Will it be born? Discovering surrogate feasibility equations for metabolic models with genetic programming

Diogo F. Oliveira*†‡, Miguel S.E. Martins*, Gonçalo M. Marques†‡, Tiago Domingos†‡,
Susana M. Vieira*, João M.C. Sousa*
*IDMEC, Instituto Superior Técnico, Universidade de Lisboa, Lisboa, Portugal
†MARETEC, Instituto Superior Técnico, Universidade de Lisboa, Lisboa, Portugal
‡Terraprima - Serviços Ambientais, Samora Correia, Portugal
{diogo.miguel.oliveira, miguelsemartins, goncalo.marques, tdomingos, susana.vieira, jmsousa}@tecnico.ulisboa.pt

***Abstract*—Metabolic models based on Dynamic Energy Budget (DEB) theory can be used to model growth, development, and reproduction for any organism and are widely applied in ecotoxicology, aquaculture, and wildlife conservation. The parameters of every DEB model must respect constraints to ensure its output is biologically consistent and realistic. To check whether the parameters allow the organism to reach birth, a set of differential equations must be solved because no closed-form expression exists; this wastes computational time and can even stall parameter estimation completely. We aim to discover a surrogate equation to determine whether a DEB model can reach birth. We develop a genetic programming (GP) classifier that evolves an explicit symbolic decision function using a DEB-motivated function set. We benchmark it against a neural network (NN) baseline on a generated dataset with sampling concentrated near the feasibility boundary. Both models achieve near-perfect discrimination, with test-set Matthews correlation coefficient values of 0.9200 for GP and 0.9577 for NN, and high recall for feasible cases (0.9785 and 0.9957, respectively). The learned symbolic equation provides a fast screening rule that can be embedded in calibration pipelines to filter out infeasible parameter sets and reduce failed simulations.**

***Index Terms*—Genetic Programming, Equation discovery, Symbolic regression, Dynamic Energy Budget theory, Bioenergetics**

I. Introduction

Life on Earth is fantastically diverse, but also remarkably similar. At the micro level, all life is built from cells that draw from a shared molecular toolkit of membranes, ribosomes, and proteins encoded by DNA and RNA that store information and convert resources into usable energy. At the macro level, this shared cellular machinery scales up, and all organisms must obtain energy and resources from their surroundings to use for growth, development, and reproduction. Regardless of the scale, these transformations are always constrained by the laws of physics.

The similarities among all life forms are what underpin metabolic theories: they aim to describe organisms under a common framework. Among metabolic theories, Dynamic Energy Budget (DEB) theory has emerged as the most general and widely applicable [1], [2]. Through a system of differential equations, DEB theory describes how an organism grows, develops, and reproduces, from conception until death, as a response to environmental forcings such as food and temperature. DEB theory is grounded in the physical principles of mass and energy conservation, which makes it possible to describe any organism [3], [4]. The Add-my-Pet (AmP) collection currently open-sources DEB models for more than 7300 species curated by a board of specialists [5], [6]. DEB models have also powered applications in ecotoxicology [7], aquaculture [8], and wildlife conservation [9].

DEB models are calibrated by solving a minimization problem: find the parameters that minimize the deviation between model predictions and available species data [10]–[13]. Additionally, the parameter set must be realistic: the organism needs to grow from embryo to adulthood and then reproduce. Parameter sets that do not yield a plausible life history are infeasible and must be filtered out during parameter estimation.

For most constraints, feasibility can be checked directly through algebraic inequalities on the parameters, but for reaching birth no closed-form condition has been discovered. Instead, feasibility is typically assessed numerically by either solving the DEB differential equations or, more efficiently, by an iterative shooting method [14]. If the simulation cannot reach birth, then the parameter set is deemed infeasible. However, this numerical step can lead to instabilities or even stall the estimation completely.

In this paper, we aim to discover a symbolic equation that predicts whether a DEB model can reach birth from its parameter values. We achieve this through genetic programming (GP) [15], a symbolic regression approach that discovers explicit data-driven expressions [16]–[18]. Guided by DEB theory, we inform the structure of the GP search by selecting functions and constants inspired by the equations used in the iterative shooting procedure [19]. The data-driven equation is an interpretable surrogate classifier that can be used in calibration procedures [20].

We frame birth feasibility as a binary classification problem. We generate a dataset with Latin hypercube sampling over the parameter ranges typically found in the AmP collection

and use theory to sample near the feasibility boundary. Each parameter set is labeled by simulating the differential equations and evaluating whether birth can be reached. To assess performance, we benchmark the best GP program against a neural network classifier. We also compare both decision boundaries to the numerical boundary obtained by simulation. The code, data, and results are available at https://github.com/diogo-f-oliveira/deb-birth-prediction.

The remainder of this paper is organized as follows. Section II summarizes the DEB birth feasibility conditions and the numerical nature of the feasibility check. Section III describes dataset generation and final characteristics. Section IV presents the proposed GP classifier and the neural network baseline. Section V reports and compares results for both models and discusses the learned symbolic boundary. Section VI concludes and outlines future work.

II. Dynamic Energy Budget theory

In its standard form, a DEB model is a system of nonlinear ordinary differential equations for reserve E, structure V, maturity E_H, and reproduction buffer E_R [1]. Food is assimilated into reserve E, which is then mobilized to fuel all metabolic processes, including growth, development, maintenance, and reproduction. Growth corresponds to an increase in structure V, whereas development is the increase in maturity E_H. Maintenance has both a somatic and a maturity component, which are proportional to structure V and maturity E_H, respectively. Maturity represents accumulated physiological information, and once specific thresholds are reached, the behavior of the model changes. Before birth ($E_H < E_H^b$), the organism is an embryo and does not feed. After puberty ($E_H > E_H^p$), maturity stops increasing and the organism instead allocates reserve to the reproduction buffer E_R.

During the embryo phase, because feeding does not occur, the organism must develop using only the initial reserves E_0 provided at egg formation. This reserve must be sufficient to cover maintenance costs throughout development and to reach the maturity threshold for birth E_H^b. Kooijman [14] derived a mathematical expression for the scaled length at birth l_b by utilizing the maternal effect assumption: the scaled reserve density at birth e_b is equal to the food level or scaled function response f of the mother. The expression is a function of f and three dimensionless compound parameters: the energy investment ratio g, the scaled maturity at birth v_H^b, and the maintenance ratio k.

Given g, k, v_H^b, and f, l_b can be obtained by solving a boundary value problem for an integro-differential equation. For the special case $k = 1$, an explicit solution is available, but for general $k \neq 1$, l_b must be computed numerically. In practice, l_b can be obtained either by integrating the DEB model differential equations or, more efficiently, by solving the boundary value problem using a Newton–Raphson shooting procedure, which iteratively updates l_b until the birth boundary condition is satisfied [14].

If l_b can be obtained, feasibility is then checked by:

$$l_b < f, \tag{1}$$

$$k\, v_H^b < l_b^2 \frac{(g + l_b) f}{g + f}. \tag{2}$$

Since $0 < l_b < f$, both conditions can be combined to obtain a sufficient but not necessary condition on $k\, v_H^b$ and f:

$$k\, v_H^b < f^3. \tag{3}$$

Equations (1)–(2) are inexpensive to evaluate. However, the numerical procedure required to obtain l_b can fail for some parameter combinations, either because no admissible solution exists or due to numerical instabilities that stall parameter estimation [21]. Currently, there is no fast screening test that reliably determines feasibility directly from (v_H^b, g, k, f) without solving for l_b, which can significantly slow down DEB parameter estimation.

III. Data

A. Dataset generation

We generate a labeled dataset in which each observation $\mathbf{x}_i$ is a random sample of the set of compound parameters relevant to the birth equations: $\mathbf{x} = (v_H^b, g, k, f)$. The label y_i is binary, $y_i = 1$ if birth can be reached, and $y_i = 0$ if not, which we obtain by numerical simulation. In total, we simulate 200,000 points.

To obtain a broad coverage of the parameter space, we use Latin hypercube sampling (LHS) [22] in a unit hypercube $[0, 1]^3$ and then transform to sample g, k, and f in the following distributions:

- $g \sim \mathrm{LogUniform}(10^{-3},\, 10^{2})$,
- $k \sim \mathrm{LogUniform}(10^{-4},\, 10^{1})$,
- $f \sim \mathrm{Uniform}(0.1,\, 1.0)$.

These parameter ranges were derived from the distributions of species parameters in the AmP collection.

To concentrate the sampled points around the birth feasibility boundary, we sample v_H^b conditioned on k and f:

$$v_H^b \sim \begin{cases} \mathrm{LogUniform}\left(10^{-1} \cdot f^3,\, 10 \cdot \dfrac{f^3}{k}\right), & k \leq 1, \\ \mathrm{LogUniform}\left(10^{-1} \cdot \dfrac{f^3}{k^3},\, 10 \cdot \dfrac{f^3}{k}\right), & k > 1. \end{cases} \tag{4}$$

The bounds are derived from Eqs. (1)–(2) and the differential equations by taking limiting cases for the length at birth l_b. Multiplying and dividing by 10 promotes sampling of points on both sides of the boundary, which will help the models to learn it.

Each sample is labeled by solving the differential equations for the scaled length at birth l_b [14]. We use the function `get_lb2` from `DEBtool`, the premier software for DEB model calibration[1]. Although the function `get_lb` is faster because it uses the iterative Newton–Raphson procedure, we

[1] `DEBtool` is a MATLAB package available at https://github.com/add-my-pet/DEBtool_M.

found that it had numerical instabilities for certain parameter ranges, which would result in incorrectly classified samples.

We set a time limit of 6 seconds for each numerical simulation of l_b given the sampled (v_H^b, g, k, f). If the time limit is reached or an execution error occurs, we consider the parameter set infeasible (i.e., birth cannot be reached). Otherwise, we determine feasibility by checking if the constraints in Eqs. (1)–(2) are met.

B. Processed dataset characteristics

The mean simulation time was 0.2 s and only 0.6% of the simulations were terminated due to the time limit. Simulations that yielded infeasible parameters were on average 2.7 times slower than feasible simulations.

More than 67% of the simulations successfully found l_b; we consider the remaining 33% infeasible. Of the successful simulations, 37% were infeasible because they did not meet Eqs. (1)–(2). As such, the final dataset has class proportions of 42.3% for reaching birth and 57.7% not reaching birth.

We split the dataset into training, validation, and test sets in proportions of 80–10–10 while keeping the class proportions consistent across splits.

IV. Proposed methods

A. Genetic programming classifier

We use genetic programming (GP) to evolve a symbolic score function $s(\mathbf{x})$ from the untransformed inputs (v_H^b, g, k, f) [15]. Using the original variables preserves a directly interpretable expression in the DEB parameterization. Each individual is a syntax tree, and its score is mapped to $\hat{p}(\mathbf{x}) = \sigma(s(\mathbf{x}))$. We set the classification threshold to 0.5, meaning that $s(\mathbf{x}) > 0$ predicts birth and $s(\mathbf{x}) = 0$ defines the learned surrogate boundary.

The function set contains arithmetic operations, inversion, negation, logarithm, square and cubic roots, square, $\arctan$, $\min$, and $\max$. It is informed by the expected boundary shape and the analytical solution for $k = 1$, which contains logarithmic and inverse-tangent terms [14]. We further include $\min$ and $\max$ operations because birth feasibility has two limiting regimes: either growth is not sufficient or development is not rapid enough. To improve numerical stability, the square root is applied to the absolute value of its argument, and logarithm inputs are floored at $\epsilon = 10^{-12}$. Inversion returns zero for arguments with magnitude below ϵ, while division replaces near-zero divisors by a signed ϵ and adds ϵ to the numerator. Instead of ephemeral random constants, the terminal set includes (v_H^b, g, k, f) and $\{1, 2, 3, 1/2, 1/3, \sqrt{2}, \sqrt{3}\}$, selected from the structure of the $k = 1$ solution.

Programs are evaluated using class-balanced binary log loss with the parsimony penalty $\lambda|T|$, where $|T|$ is the number of tree nodes [23]. Evolution uses tournament selection, reproduction, crossover, and subtree, hoist, and point mutation [15], [24]. The population contains 1000 programs and evolves for 100 generations. The initial population is generated with the half-and-half method with depths between 6 and 10 [15].

TABLE I
Search ranges and selected GP hyperparameters.

Hyperparameter	Search	Selected
Tournament size	50–500 ($\Delta = 10$)	190
λ	$\mathrm{LogUniform}(10^{-5}, 10^{-3})$	1.3×10^{-4}
$p_{\mathrm{reproduction}}$	0–0.40 ($\Delta = 0.01$)	0.22
p_{mutation}	0–0.40 ($\Delta = 0.01$)	0.11
p_{subtree}		0.085
p_{hoist}	$\mathrm{Dirichlet}(1, 1, 1)$	0.005
p_{point}		0.020

The algorithm was implemented with `gplearn` 0.4.3 in Python 3.10. We evaluated 100 HyperOpt configurations [25] with `ray.tune`, using one run per configuration (the random seed was set to 42). Each configuration was fitted on the training split and ranked by its macro-F1 score on the validation split. The tuned ranges and selected values are given in Table I.

The Dirichlet distribution samples nonnegative shares for subtree, hoist, and point mutation that sum to 1. Multiplying these shares by the selected p_{mutation} gives the three mutation probabilities reported in Table I. The crossover probability is the remaining probability, $p_{\mathrm{crossover}} = 1 - p_{\mathrm{reproduction}} - p_{\mathrm{mutation}} = 0.67$.

B. Neural network baseline

To obtain a baseline for performance, we train a feed-forward neural network that takes the same input vector $\mathbf{x} = (v_H^b, k, g, f)$. Before being passed to the network, we apply a logarithm transformation followed by standardization to all inputs. The parameters have ranges that can span more than 10 orders of magnitude, so the logarithm transformation is crucial to stabilize gradients.

The network has three hidden layers with 32 units each. We use ReLU activation functions and dropout with probability 0.2 after each hidden layer. The output layer has one unit and produces a logit, which is mapped to a probability with a sigmoid.

The network is implemented in PyTorch in Python 3.10 and trained for 100 epochs with a batch size of 64. As with the GP, we use the weighted binary cross-entropy loss $\mathcal{L}_{BCE}$ to account for class imbalance. We employ the AdamW optimizer and set the learning rate to 10^{-4} and the weight decay to 10^{-3}.

V. Results and Discussion

A. Classification performance

Table II presents test-set metrics for both the genetic programming model and the neural network baseline. The class proportions are moderately imbalanced (42.3% reaches birth compared with 57.7% that does not), so we report overall and per-class metrics, including accuracy, AUROC, and average precision.

In calibration procedures, the proposed models can be used to screen for infeasible parameter values without solving the differential equations. A correct screening can save calibration time and avoid cases where the simulation stalls due to

TABLE II
TEST SET METRICS FOR THE GENETIC PROGRAMMING MODEL AND NEURAL NETWORK BASELINE.

Class	Metric	GP	NN
Birth is reached	Precision	0.9310	0.9551
	Recall	0.9785	0.9957
	F1-score	0.9541	0.9750
Birth is not reached	Precision	0.9836	0.9968
	Recall	0.9469	0.9657
	F1-score	0.9649	0.9810
Overall	Accuracy	0.9602	0.9784
	AUROC	0.9948	0.9994
	MCC	0.9200	0.9577

numerical instabilities. However, rejecting a parameter set that is feasible may be more detrimental, since that parameter set may improve the fit between the data and the DEB model or even be the best parameter set for the species. For this reason, we prioritize maximizing recall for the class *Birth is reached*. Nevertheless, since filtering out infeasible cases will lead to efficiency improvements, high precision for *Birth is reached* (accepted sets should be feasible) and high recall for *Birth is not reached* (infeasible sets should be filtered early) should also be maximized.

Overall, both models achieve very high performance with the neural network outperforming the best program found with genetic programming by less than 3% on all metrics. For the critical metric, recall for the class *Birth is reached*, the NN achieves a value of 0.9957 versus 0.9785 for the GP, indicating that both could be employed successfully as screening methods.

Precision for the class *Birth is reached* is 0.9551 for the neural network versus 0.9310 for the evolved program, whereas for the *Birth is not reached* class, recall is 0.9657 for the NN versus 0.9469 for the evolved program. These results demonstrate that both models perform very well in filtering out infeasible parameter sets and could lead to efficiency improvements if implemented in calibration procedures.

Figure 1 shows the precision–recall curve for the class *Birth is reached*. Both models maintain very high precision over a wide range of recall values, with the NN dominating GP for high recall values. The corresponding average precision values are 0.9926 (GP) and 0.9974 (NN).

Overall accuracy is 0.9602 for the GP and 0.9784 for the NN. For context, a trivial predictor that always predicts the majority class of *Birth is not reached* would achieve an accuracy of 0.577, so both models provide substantial predictive value beyond the class imbalance.

The NN's modest performance advantage likely reflects differences in model representation and optimization. The NN adjusts a comparatively large set of continuous weights through gradient-based optimization, allowing it to approximate the numerically defined feasibility boundary without expressing it as a compact composition of prescribed functions. Moreover, log transformation and standardization supported NN training by placing the inputs on comparable numerical scales, an

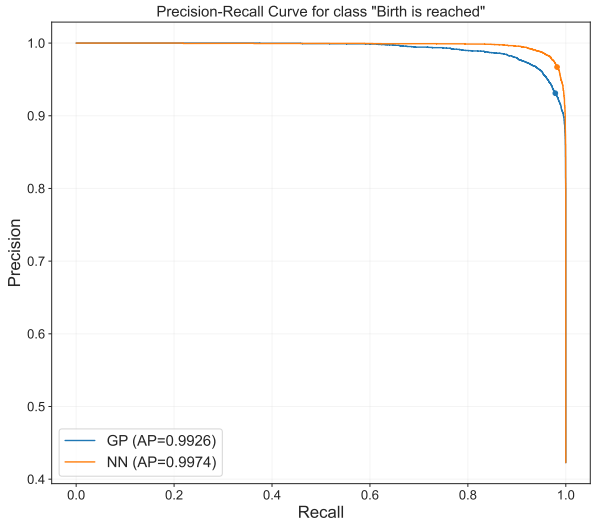

Fig. 1. Precision–recall curve for the class *Birth is reached*.

important preprocessing step because DEB parameters span several orders of magnitude [21]. In contrast, the GP is restricted to the selected primitive and fixed-constant sets and is subject to parsimony pressure, which deliberately trades some predictive flexibility for an explicit and interpretable decision function. The performance gap may therefore be narrowed through refinements to the evolutionary optimization strategy that improve search efficiency while retaining the same GP representation or by exploring alternative model representations.

B. Best program characteristics

The best predictive equation found by the genetic programming model had a depth of 22 and length of 141. After evolution, the expression was simplified with SymPy for presentation, reducing its length to 120 while retaining a depth of 22. The expression is too long to report here, but some traits can be observed about the score function $s(\mathbf{x})$. It increases with f and decreases with v_H^b, reflecting a trade-off between energy availability and the maturity requirement for birth. Additionally, nested $\min(\cdot)$ and $\max(\cdot)$ operators make the score function piecewise, akin to selecting between alternative limiting regimes.

After implementation in MATLAB, evaluating the program on the test set on a per-sample basis takes, on average, 0.7 $\pm$ 1.3 µs. Compared with the mean simulation time of 0.2 s per parameter vector, this inference time per parameter vector corresponds to a speedup of approximately 2.9×10^5, or 5.46 orders of magnitude. Typically, the feasibility of each parameter set is checked individually; however, the evolved expression can be vectorized, leading to further speedups for population-based calibration algorithms [26].

C. Feasibility boundary comparison

Figure 2 compares simulated birth feasibility regions with decision boundaries predicted by genetic programming (GP) and a neural network (NN) for four fixed slices of (k, f).

Panel (a) corresponds to the theoretical case for which a closed-form solution exists: $k = 1$. Here, both conditions in Equations (1)–(2) reduce to the constraint given in Equation 2. Both approximate the boundary well, but not perfectly, with the evolved program performing better for smaller values of g and the NN performing better for larger values of g.

Panel (b) corresponds to the most common parameter values in the Add-my-Pet collection, $k = 0.3$ and $f = 1$. Here, both models closely follow the simulated boundary and mostly remain within the theoretical bounds. For a smaller food level (panel (c), $f = 0.3$ and $k = 0.3$), the agreement is similar, although the evolved program overshoots the theoretical bound more than the neural network.

For the high-k regime in panel (d), the GP program performs worse and does not approach the bounds. However, the NN produces an additional contour for small values of v_H^b and g where it predicts that the DEB model cannot reach birth. In this region, both models extrapolate because the dataset does not cover these parameter ranges; the numerical solver therefore remains the authoritative feasibility check. Although both boundaries are approximate, the neural network closely follows the simulated boundary within the dataset ranges. In contrast, while the evolved program performs slightly worse than the neural network, it exhibits consistent behavior even when extrapolating. Species with $k > 1$ are very rare in the Add-my-Pet collection, as this regime may be incompatible with other biological constraints, which mitigates this issue.

VI. Conclusions and future work

Reaching birth is a necessary feasibility requirement of any organism. As such, any parameterized DEB model must also be able to do so. Checking whether birth can be reached typically requires an iterative numerical procedure that can be unstable or even halt the estimation indefinitely. To improve the reliability and efficiency of DEB model calibration, we developed fast surrogate models that predict whether birth can be reached directly from the compound parameters (g, k, v_H^b, f). Genetic programming discovers a symbolic predictive equation built from theory-informed functions and constants. To benchmark performance, we trained a neural network classifier.

On a generated dataset of birth simulations, both models achieve high performance, with a Matthews correlation coefficient of at least 0.92. The neural network is slightly more accurate, but GP remains competitive while offering an explicit closed-form expression. In practice, either surrogate can be used as a screening step to reject infeasible candidates before running the full solver, thereby substantially reducing failed simulations and improving the efficiency of DEB model calibration.

For future work, the GP algorithm could be improved by implementing expression simplification during evolution and adopting grammar-based techniques [27], including a grammar that encodes the analytical bounds used to generate the dataset. The approach we developed can also be extended to other feasibility constraints in DEB models, such as reaching metamorphosis in fish or pupation in insect models. We will also work to incorporate our feasibility screening models into `DEBtool` to improve the existing calibration procedure for all DEB modelers.

Finally, beyond filtering, these feasibility models can be integrated into learning pipelines that directly predict DEB parameters from species data [21]. The neural model provides a differentiable feasibility penalty, and smoothed variants of GP expressions can play the same role, encouraging feasible parameter predictions during training.

Acknowledgments

This work was supported by the Portuguese Foundation for Science and Technology (FCT), through IDMEC, under LAETA, project UIDB/50022/2020, and by FCT/MCTES (PIDDAC) through projects UIDB/50009/2025, UIDP/50009/2025, and LA/P/0083/2020. This work was also supported by Fundo Europeu de Desenvolvimento Regional through project AVALON (COMPETE2030-FEDER-02288900). The work of Diogo F. Oliveira was supported by the PhD Studentship 2022.14030.BDANA from FCT.

References

[1] B. Kooijman, *Dynamic Energy Budget Theory for Metabolic Organisation*. Cambridge: Cambridge University Press, 3rd ed., 2010.

[2] M. Jusup and M. R. Kearney, "The untapped power of a general theory of organismal metabolism," Aug. 2024. arXiv:2408.13998 [q-bio].

[3] T. Sousa, T. Domingos, J.-C. Poggiale, and S. a. L. M. Kooijman, "Dynamic energy budget theory restores coherence in biology," *Philosophical Transactions of the Royal Society B: Biological Sciences*, vol. 365, pp. 3413–3428, Nov. 2010.

[4] S. A. L. M. Kooijman, M. R. Kearney, N. Marn, T. Sousa, T. Domingos, R. Lavaud, C. Récapet, T. Klanjšček, T. T. Yeuw, G. M. Marques, L. Pecquerie, and K. Lika, "From formulae, via models to theories: Dynamic Energy Budget theory illustrates requirements," *Ecological Modelling*, vol. 497, p. 110869, Nov. 2024.

[5] Add-my-Pet Board of Curators, "Add-my-Pet Collection." https://www.bio.vu.nl/thb/deb/deblab/add_my_pet/. Accessed: 2026-01-26.

[6] G. M. Marques, S. Augustine, K. Lika, L. Pecquerie, T. Domingos, and S. A. L. M. Kooijman, "The AmP project: Comparing species on the basis of dynamic energy budget parameters," *PLOS Computational Biology*, vol. 14, p. e1006100, May 2018.

[7] T. Jager, E. H. W. Heugens, and S. A. L. M. Kooijman, "Making Sense of Ecotoxicological Test Results: Towards Application of Process-based Models," *Ecotoxicology*, vol. 15, pp. 305–314, Apr. 2006.

[8] T. Y. Tan, M. C. Miraldo, R. F. C. Fontes, and F. S. Vannucchi, "Assessing bivalve growth using bio-energetic models," *Ecological Modelling*, vol. 473, p. 110069, Nov. 2022.

[9] U. Dajčman, U. Enriquez-Urzelai, and A. Žagar, "Microclimate variability impacts the coexistence of highland and lowland ectotherms," *Journal of Animal Ecology*, vol. 94, no. 5, pp. 999–1013, 2025.

[10] K. Lika, M. R. Kearney, V. Freitas, H. W. van der Veer, J. van der Meer, J. W. M. Wijsman, L. Pecquerie, and S. A. L. M. Kooijman, "The "covariation method" for estimating the parameters of the standard Dynamic Energy Budget model I: Philosophy and approach," *Journal of Sea Research*, vol. 66, pp. 270–277, Nov. 2011.

[11] G. M. Marques, K. Lika, S. Augustine, L. Pecquerie, and S. A. L. M. Kooijman, "Fitting multiple models to multiple data sets," *Journal of Sea Research*, vol. 143, pp. 48–56, Jan. 2019.

[12] D. F. Oliveira, G. M. Marques, N. Carolino, J. Pais, J. M. C. Sousa, and T. Domingos, "A multi-tier methodology for the estimation of individual-specific parameters of DEB models," *Ecological Modelling*, vol. 494, p. 110779, Aug. 2024.

[13] D. F. Oliveira, A. Sulc, E. L. Moan, E. Donati, E. Klagkou, M.-J. Lagunes, U. Dajčman, and T. Y. Tan, "How to build Dynamic Energy Budget models: Practical guidelines for model development," *Ecological Modelling*, vol. 521, p. 111703, Nov. 2026.

[14] S. A. L. M. Kooijman, "What the hen can tell about her eggs: egg development on the basis of energy budgets," *Journal of Mathematical Biology*, vol. 23, pp. 163–185, Feb. 1986.

[15] J. R. Koza, "Genetic programming as a means for programming computers by natural selection," *Statistics and Computing*, vol. 4, pp. 87–112, June 1994.

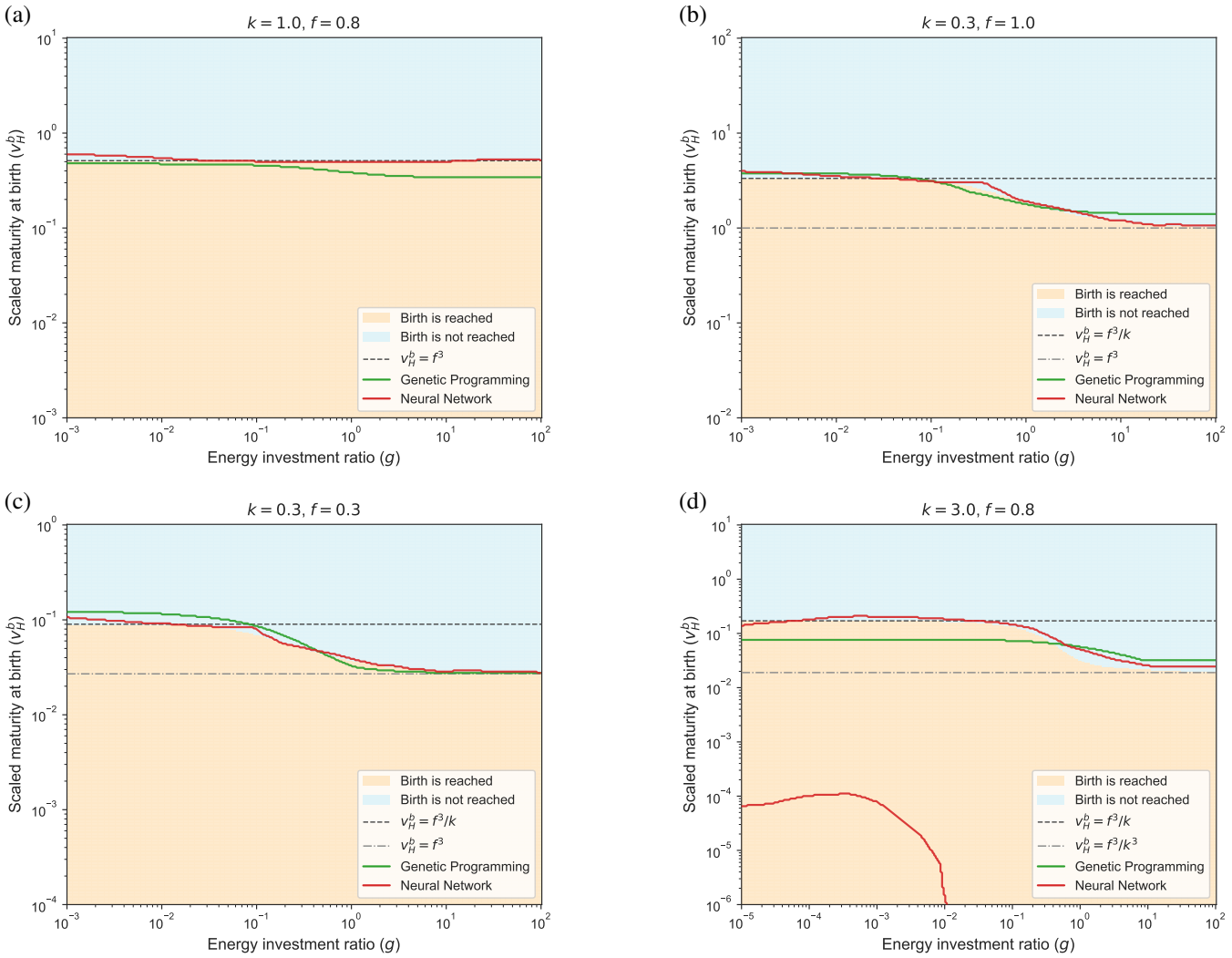

Fig. 2. Birth feasibility regions and learned decision boundaries for fixed (k, f) slices of the DEB parameter space. Shaded areas indicate the simulated outcome of the DEB model on a log-uniform (g, v_H^b) grid (orange: birth is reached; blue: birth is not reached). Solid curves show the decision boundaries predicted by the genetic programming (green) and neural network (red) classifiers. Dashed reference lines indicate the analytical bounds for v_H^b in Eq. (4).

[16] M. Schmidt and H. Lipson, "Distilling Free-Form Natural Laws from Experimental Data," *Science*, vol. 324, pp. 81–85, Apr. 2009.

[17] D. Angelis, F. Sofos, and T. E. Karakasidis, "Artificial Intelligence in Physical Sciences: Symbolic Regression Trends and Perspectives," *Archives of Computational Methods in Engineering*, vol. 30, pp. 3845–3865, July 2023.

[18] Y. Wang, N. Wagner, and J. M. Rondinelli, "Symbolic regression in materials science," *MRS Communications*, vol. 9, pp. 793–805, Sept. 2019.

[19] Q. Lu, J. Ren, and Z. Wang, "Using Genetic Programming with Prior Formula Knowledge to Solve Symbolic Regression Problem," *Computational Intelligence and Neuroscience*, vol. 2016, no. 1, p. 1021378, 2016.

[20] Y. Mei, Q. Chen, A. Lensen, B. Xue, and M. Zhang, "Explainable Artificial Intelligence by Genetic Programming: A Survey," *IEEE Transactions on Evolutionary Computation*, vol. 27, pp. 621–641, June 2023.

[21] D. F. Oliveira, G. M. Marques, F. M. P. Santos, L. Pecquerie, J. M. C. Sousa, and T. Domingos, "Reliable machine learning initialization methods for the calibration of Dynamic Energy Budget models," *Ecological Informatics*, p. 103624, Jan. 2026.

[22] M. D. McKay, R. J. Beckman, and W. J. Conover, "A Comparison of Three Methods for Selecting Values of Input Variables in the Analysis of Output from a Computer Code," *Technometrics*, vol. 21, no. 2, pp. 239–245, 1979.

[23] C. Gagné, M. Schoenauer, M. Parizeau, and M. Tomassini, "Genetic Programming, Validation Sets, and Parsimony Pressure," Jan. 2006. arXiv:cs/0601044.

[24] P. G. Espejo, S. Ventura, and F. Herrera, "A Survey on the Application of Genetic Programming to Classification," *IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews)*, vol. 40, pp. 121–144, Mar. 2010.

[25] J. Bergstra, D. Yamins, and D. Cox, "Making a Science of Model Search: Hyperparameter Optimization in Hundreds of Dimensions for Vision Architectures," in *Proceedings of the 30th International Conference on Machine Learning*, pp. 115–123, PMLR, Feb. 2013.

[26] J. F. Robles, M. Chica, R. Filgueira, A. Agüera, and S. Damas, "Multi-Calib4DEB: A toolbox exploiting multimodal optimisation in Dynamic Energy Budget parameters calibration," Jan. 2023. arXiv:2301.07548 [cs].

[27] R. I. McKay, N. X. Hoai, P. A. Whigham, Y. Shan, and M. O'Neill, "Grammar-based Genetic Programming: a survey," *Genetic Programming and Evolvable Machines*, vol. 11, pp. 365–396, Sept. 2010.